

Софтуерна архитектура. Проектиране и документирание на софтуерни архитектури

Ивайло Андреев + DeepSeek-V3

13 юни 2026

Съдържание

1	Понятие за софтуерна архитектура. Структури и изгледи. Необходимост. Влияние	3
1.1	Понятие за софтуерна архитектура	3
1.2	Структури и изгледи (Structures and Views)	3
1.2.1	Модел 4+1	4
1.3	Необходимост от софтуерни архитектури	4
1.4	Влияние на архитектурата върху проекта и организацията (Architecture-Business Cycle)	4
2	Архитектурни стилове	6
2.1	Многослоен стил (Layered Architecture)	6
2.2	Модел-Изглед-Контролер (Model-View-Controller, MVC)	6
2.3	Поточни архитектури (Pipe-and-Filter Architecture)	7
2.4	Архитектура, ориентирана към услуги (Service-Oriented Architecture, SOA)	7
3	Проектиране на софтуерната архитектура	9
3.1	Процес за проектиране	9
3.2	Архитектурни драйвери (Architectural Drivers)	9
3.3	ADD (Attribute Driven Design)	9
3.4	Тактики (архитектурни решения) за постигане на качествени показатели	10
3.4.1	Тактики за производителност (Performance)	10
3.4.2	Тактики за улесняване на тестването (Testability)	10
3.4.3	Тактики за сигурност (Security)	10
3.4.4	Тактики за удобство при употреба (Usability)	11
3.4.5	Тактики за изправност (Reliability)	11
3.4.6	Тактики за изменяемост (Modifiability)	11
4	Документирание на софтуерната архитектура	12
4.1	Предназначение на документацията	12
4.2	Основен принцип на документирание	12
4.3	Съдържание и структура на документацията	12
4.4	Документирание на архитектурен изглед	13

4.5	Специфични архитектурни изгледи	13
4.5.1	Модулна декомпозиция (Module Decomposition View)	13
4.5.2	Изглед на процесите (Process View)	13
4.5.3	Изглед на разположението (Deployment View)	14

1 Понятие за софтуерна архитектура. Структури и изгледи. Необходимост. Влияние

1.1 Понятие за софтуерна архитектура

Според Software Engineering Institute (SEI):

“Софтуерната архитектура на дадена програма или изчислителна система е структурата или структурите на системата, които включват софтуерните елементи, външно видимите свойства на тези елементи и връзките между тях.”

С прости думи, софтуерната архитектура представлява:

- **Фундаменталната организация** на системата.
- **Съвкупност от архитектурни структури** (структури и изгледи).
- **Елементи, връзки и ограничения** между тях.
- **Ключови проектни решения**, които оформят системата.
- **Основата за развитие и поддръжка** на системата през целия ѝ живот.

1.2 Структури и изгледи (Structures and Views)

Важно е да се прави разлика между **структура** и **изглед**:

- **Структура (Structure):** Това е *реалната организация* на системата – как елементите са свързани помежду си. Една система има много структури.
- **Изглед (View):** Това е *представяне на дадена структура* за определена аудитория (например архитекти, разработчици, клиенти). Нито един изглед не описва цялата архитектура.

Архитектурните изгледи се делят на три основни групи:

1. Модулни изгледи (Module Views):

- Декомпозиция на модулите.
- Употреба на модулите.
- Изглед на слоевете (многослоен).
- Йерархия на класовете.

2. Изгледи на процесите (Process Views):

Показват изпълняваните процеси, нишки и тяхното взаимодействие.

3. Изгледи на разположението (Allocation/Deployment Views):

- Изглед на внедряването (Deployment).
- Файлов изглед.
- Разпределение на работата.

1.2.1 Модел 4+1

Един популярен подход за представяне на архитектурата е моделът **4+1** (на Philippe Kruchten):

1. **Логически изглед (Logical View):** Основните класове, компоненти и под-системи.
2. **Изглед на процесите (Process View):** Изпълняваните процеси, нишки и тяхното взаимодействие.
3. **Изглед на кода (Development View):** Организацията на програмния код (модули, библиотеки, пакети).
4. **Физически изглед (Deployment View):** Разполагането на софтуера върху хардуер (сървъри, клиенти, устройства).
5. **(+1) Сценарии на употреба (Use Case/Scenarios):** Примери за реална работа на системата, които свързват останалите четири изгледа.

1.3 Необходимост от софтуерни архитектури

Архитектурата е необходима поради няколко основни причини:

- **Управление на сложността (Complexity Management):** Разделя системата на по-малки, разбираеми части. Позволява по-лесно разбиране на цялостната система.
- **Основа за анализ (Foundation for Analysis):** Служи като база за анализ на качеството, риска, цената и сроковете още в ранните етапи на разработката.
- **Комуникация (Communication):** Осигурява общ език и референтна рамка между различните заинтересовани страни – архитекти, разработчици, тестери, клиенти и мениджъри.
- **Повторна употреба (Reuse):** Позволява използването на доказани архитектурни шаблони и решения, което спестява време и ресурси.

1.4 Влияние на архитектурата върху проекта и организацията (Architecture-Business Cycle)

Връзката между бизнеса и архитектурата е двупосочна (цикъл):

- **Бизнесът влияе върху архитектурата чрез:**
 - **Изисквания** (функционални и качествени).
 - **Бюджет** (ограничения върху ресурсите).
 - **Срокове** (време за разработка).
- **Архитектурата влияе върху бизнеса и организацията чрез:**
 - **Структурата на екипа** (екипите често следват архитектурната модулна структура).
 - **Управление на риска** (рисковете са идентифицирани на архитектурно ниво).

- **Разходите и поддръжката** (добрата архитектура намалява разходите за поддръжка и улеснява добавянето на нови функции).

2 Архитектурни стилове

Архитектурен стил е набор от:

- **Компоненти** – изпълними единици, които извършват изчисления (напр. модули, класове, услуги).
- **Конектори** – механизми за комуникация между компонентите (напр. синхронни/асинхронни извиквания, съобщения).
- **Ограничения** – правила за това как компонентите могат да се комбинират.
- **Правила за взаимодействие** – семантика на комуникацията.

2.1 Многослоен стил (Layered Architecture)

При този стил системата се организира в йерархия от слоеве. **Всеки слой предоставя услуги само на слоя над себе си и използва услуги само от слоя под себе си.**

Предимства:

- **Разделяне на отговорностите** – всеки слой има ясна роля.
- **Скриване на информацията** – вътрешната реализация на слой е скрита.
- **Висока кохезия** – елементите в един слой са силно свързани функционално.
- **Лесна поддръжка и тестване** – слоевете могат да се тестват и заменят независимо.

Недостатъци:

- **Допълнителни зависимости** – понякога се налага „заобикаляне“ на слоевете, което нарушава модела.
- **По-ниска производителност** – заявката преминава през няколко слоя, което добавя забавяне.

2.2 Модел-Изглед-Контролер (Model-View-Controller, MVC)

MVC разделя системата на три взаимносвързани компонента:

1. **Model (Модел):** Отговаря за данните и бизнес логиката. Уведомява View-тата при промяна.
2. **View (Изглед):** Отговаря за визуализацията на данните (потребителския интерфейс).
3. **Controller (Контролер):** Обработва входните действия от потребителя, променя модела и избира подходящ изглед.

Предимства:

- **Разделяне на отговорностите** – ясно разделение на UI, логика и контрол.
- **Множество изгледи** – един и същ модел може да има различни изгледи.
- **Добра поддръжка** – промените в един компонент не влияят драстично на другите.

Недостатъци:

- **Допълнителна сложност** – за прости приложения MVC може да е излишен.
- **Допълнителен код** – изисква писане на повече код в сравнение с неструктуриран подход.

2.3 Поточни архитектури (Pipe-and-Filter Architecture)

Системата се състои от последователни компоненти, наречени **филтри (filters)**, които:

- Получават входни данни.
- Обработват ги.
- Подават резултата към следващия компонент по **тръби (pipes)**.

Пример: Input → Filter1 → Filter2 → Filter3 → Output

Предимства:

- **Повторна употреба** – филтрите могат да се пренареждат и използват в различни конфигурации.
- **Паралелизъм** – филтрите могат да работят паралелно, ако тръбите буферират данните.
- **Независими компоненти** – филтрите нямат знание един за друг.

Недостатъци:

- **Слаба производителност** – при много филтри и големи данни, серийната обработка може да е бавна.
- **Трудно споделяне на глобални данни** – филтрите са изолирани, което затруднява достъпа до споделено състояние.

2.4 Архитектура, ориентирана към услуги (Service-Oriented Architecture, SOA)

SOA е архитектурен стил, при който функционалността се предоставя като **услуги (services)** през мрежа. Основни участници:

- **Доставчик на услуги (Service Provider):** Създава и публикува услуги.

- **Регистър на услуги (Service Registry):** Каталог, където услугите се регистрират и откриват.
- **Консуматор на услуги (Service Consumer):** Открива и използва услуги.

Основни операции в SOA:

1. **Publish (Публикуване):** Доставчикът регистрира услугата в регистъра.
2. **Find (Откриване):** Консуматорът търси услуга в регистъра.
3. **Bind (Свързване):** Консуматорът се свързва към услугата и я използва.

Характеристики:

- **Слаба свързаност (Loosely coupled):** Услугите са независими една от друга.
- **Повторна употреба (Reusability):** Услугите могат да се използват от множество приложения.
- **Оперативна съвместимост (Interoperability):** Услугите комуникират чрез стандартни протоколи (напр. HTTP, SOAP, REST).

3 Проектиране на софтуерната архитектура

3.1 Процес за проектиране

Проектирането на архитектура е **цикличен процес**, който включва следните основни стъпки:

1. **Анализ на изискванията:** Разбиране на функционалните и качествените изисквания, както и на бизнес ограниченията.
2. **Избор на архитектурен стил:** Избор на подходящ стил (или комбинация от стилове) според анализа.
3. **Декомпозиция:** Разделяне на системата на по-малки модули, компоненти или подсистеми.
4. **Дефиниране на интерфейси:** Определяне на това как компонентите ще комуникират помежду си.
5. **Оценка:** Проверка дали архитектурните решения удовлетворяват изискванията.

3.2 Архитектурни драйвери (Architectural Drivers)

Това са основните фактори, които управляват архитектурните решения:

- **Функционални изисквания:** Какво трябва да прави системата (напр. „системата трябва да генерира месечен отчет“).
- **Качествени показатели (Quality Attributes):** Производителност, сигурност, надеждност, използваемост, изменяемост и др.
- **Бизнес ограничения:** Срокове, бюджет, задължителни технологии, съответствие със стандарти.

3.3 ADD (Attribute Driven Design)

ADD е метод за проектиране на архитектурата, който поставя качествените показатели в центъра на процеса. Основните стъпки на ADD са:

1. Избор на елемент за декомпозиция (например цялата система или голям модул).
2. Избор на архитектурни драйвери за този елемент (кои качествени показатели са най-важни).
3. Избор на архитектурен стил или шаблон, който адресира тези драйвери.
4. Създаване на подмодули (декомпозиция на елемента).
5. Дефиниране на интерфейсите на подмодулите.
6. Проверка на решенията и повторение на процеса за подмодулите.

3.4 Тактики (архитектурни решения) за постигане на качествени показатели

Тактика е архитектурно решение, което влияе директно върху постигането на даден качествен показател. Наборът от тактики образува **архитектурна стратегия**.

3.4.1 Тактики за производителност (Performance)

- **Намаляване на изискванията:**
 - По-добри алгоритми.
 - Кеширане на данни.
 - Намаляване на overhead (например използване на ефективни протоколи).
 - Ограничаване на времето за изпълнение (таймаути).
- **Управление на ресурсите:**
 - Паралелизъм (многопоточност, разпределени изчисления).
 - Излишък (redundancy) – кешове, реплики.
 - Добавяне на повече ресурси (памет, процесорно време).
- **Арбитраж (Scheduling):**
 - FIFO (First-In-First-Out).
 - Приоритети на заявките.

3.4.2 Тактики за улесняване на тестването (Testability)

- **Record & Replay:** Записване на входни данни и повтаряне на тестовете.
- **Интерфейс срещу реализация (Interface vs Implementation):** Тестване чрез абстрактни интерфейси.
- **Специален тестов интерфейс (Test Interface):** Предоставяне на специални методи за тестване.
- **Мониторинг:** Наблюдение на състоянието на системата по време на тестове.

3.4.3 Тактики за сигурност (Security)

- **Устояване на атаки:**
 - Authentication (удостоверяване на самоличност).
 - Authorization (оторизация – контрол на достъпа).
 - Encryption (криптиране на данни).
 - Integrity (проверка на целостта на данните).
- **Възстановяване след атака:**
 - Възстановяване на състоянието (restore).
 - Одитен журнал (Audit trail) за проследяване на действията.

3.4.4 Тактики за удобство при употреба (Usability)

- **По време на изпълнение (за крайния потребител):**
 - Обратна връзка (Feedback) за действията на потребителя.
 - Отмяна (Undo) и анулиране (Cancel) на операции.
 - Множество изгледи (Multiple Views) на данните.
 - Баланс между автоматични действия и инициатива на потребителя.
- **Насочени към разработчика на интерфейса:** API-та и инструменти за лесно изграждане на потребителски интерфейси.

3.4.5 Тактики за изправност (Reliability)

- **Откриване и предпазване от откази:**
 - Ping / Echo – проверка дали компонент е жив.
 - Heartbeat – периодични сигнали за състояние.
 - Exceptions – обработка на изключения.
- **Отстраняване на откази:**
 - Активен излишък (Active redundancy) – всички резервни компоненти работят паралелно.
 - Пасивен излишък (Passive redundancy) – резервен компонент се активира при отказ.
 - Извеждане от употреба на дефектния компонент.
 - Следене на процесите (Process Monitoring).
- **Повторно въвеждане в употреба:**
 - Паралелна работа (shadow mode) – новата версия работи паралелно със старата.
 - Контролни точки и връщане (Checkpoint / Rollback).

3.4.6 Тактики за изменяемост (Modifiability)

- **Локализиране на промените:**
 - Поддържане на семантична свързаност (cohesion).
 - Очакване на промени (предвиждане кои части вероятно ще се променят).
 - Ограничаване на обхвата на опциите.
- **Предотвратяване на ефекта на вълната (Ripple Effect):**
 - Скриване на информация (Information Hiding).
 - Ограничена комуникация между модулите.
 - Използване на посредници (Mediators).
- **Отлагане на свързването (Defer Binding):**
 - Конфигурационни файлове.
 - Компоненти с възможност за „plug-and-play“.
 - Полиморфизъм и динамично зареждане.

4 Документиране на софтуерната архитектура

4.1 Предназначение на документацията

Архитектурната документация служи за:

- **Комуникация** между всички заинтересовани страни.
- **Обучение** на нови членове на екипа.
- **Анализ** на архитектурата и вземане на информирани решения.
- **Поддръжка** и еволюция на системата.
- **Съхраняване на знания** за системата (knowledge management).

4.2 Основен принцип на документиране

Архитектурата се документира чрез множество изгледи. Нито един изглед не е достатъчен, за да опише цялата архитектура. Различните изгледи са насочени към различни аудитории и показват различни аспекти на системата.

4.3 Съдържание и структура на документацията

Една добра архитектурна документация трябва да съдържа следните елементи:

Основна информация:

- **Цели** на документа.
- **Контекст** на системата (как взаимодейства с външния свят).
- **Терминология** – дефиниции на основните понятия.

Архитектурни изгледи (Architectural Views):

- Модулни изгледи (например декомпозиция, слоеве).
- Изгледи на процесите (комуникация между процеси/нишки).
- Изгледи на разположението (хардуер, мрежа).

Отвъд изгледите (Beyond Views):

- **Архитектурни решения** – защо са взети определени решения.
- **Ограничения** – технологии, стандарти, бюджет.
- **Обосновка (Rationale)** – аргументите зад архитектурния дизайн.

4.4 Документиране на архитектурен изглед

Всеки архитектурен изглед трябва да съдържа следните елементи:

1. **Основно представяне** – например UML диаграма.
2. **Елементи и връзки** – описание на компонентите и конекторите.
3. **Контекст** – как този изглед се свързва с останалите.
4. **Варианти на използване** – как изгледът поддържа различни сценарии.
5. **Архитектурна обосновка** – защо този изглед изглежда така.
6. **Терминологичен речник** – специфични термини за изгледа.
7. **Допълнителна информация** – препратки, версии.

4.5 Специфични архитектурни изгледи

4.5.1 Модулна декомпозиция (Module Decomposition View)

- Показва йерархичното разделяне на системата на модули.
- Връзките са от типа „X е под-модул на Y“.
- Всеки модул има ясно дефинирана отговорност, предоставя интерфейс и скрива вътрешната си реализация.

4.5.2 Изглед на процесите (Process View)

- Елементите са процеси или нишки.
- Конекторите са комуникационни, синхронизационни или блокиращи операции.
- Показва паралелно изпълнение, времеви ограничения и комуникация между компонентите.

За представяне на изгледа на процесите най-често се използват следните UML диаграми:

Таблица 1: Избор на UML диаграма за представяне на изглед на процесите

Диаграма	Кога се използва	Какво показва
Sequence Diagram	Когато има комуникация между процеси и ясен сценарий.	Последователност от съобщения между процеси във времето.
Activity Diagram	Когато има поток от действия и разклонения.	Последователност от действия и паралелни потоци.
State Machine Diagram	Когато системата има състояния и реагира на събития.	Състояния и преходи между тях.

4.5.3 Изглед на разположението (Deployment View)

- Показва как софтуерните елементи се разполагат върху физическата среда (хардуер).
- Описва връзката между софтуер и хардуер.
- Представа комуникацията между устройства и среди.

Този изглед се създава след като вече имаме модулна декомпозиция и изглед на процесите, тъй като тогава компонентите са ясно дефинирани.

Заклучение

Софтуерната архитектура е фундаментът, върху който се изгражда всяка сложна софтуерна система. Тя включва структури и изгледи, които помагат за управление на сложността, комуникация и анализ. Различните архитектурни стилове – многослоен, MVC, поточен, SOA – предлагат доказани решения за различни проблемни области. Проектирането на архитектурата е цикличен процес, управляван от функционални и качествени изисквания, като тактиките за качествени показатели (производителност, сигурност, надеждност и др.) насочват конкретните архитектурни решения. Накрая, документацията чрез множество изгледи осигурява дълготрайна комуникация, поддръжка и еволюция на системата. Разбирането и правилното прилагане на тези концепции е от решаващо значение за успеха на всеки софтуерен проект.